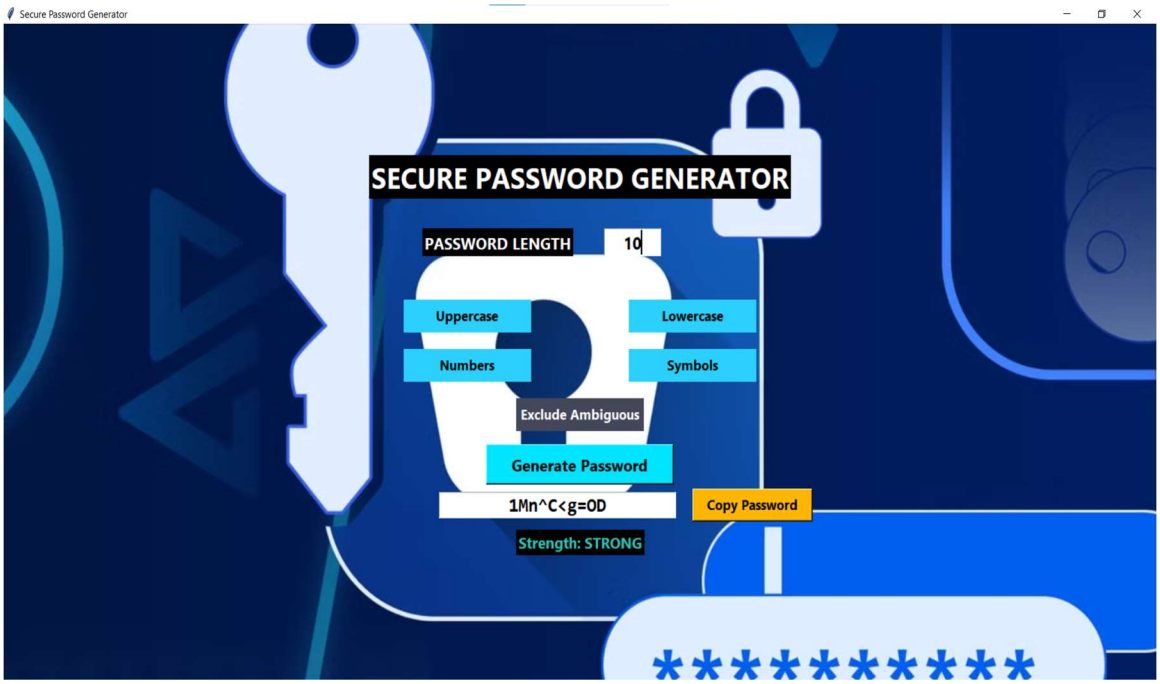
242BCA24 – Joel Anthony N

Secure Password Generator - Software Documentation



1. Introduction

The Secure Password Generator is a Python-based application that helps users create strong and secure passwords. The program allows users to customize password generation by selecting different character types such as uppercase letters, lowercase letters, numbers, and symbols. The system also evaluates the strength of the generated password and displays whether it is Weak, Moderate, or Strong. This helps users understand the security level of their passwords.

The application uses a graphical user interface (GUI) built with Tkinter so that users can interact with the program easily.

2. Objective of the Project

• Generate secure passwords automatically.

• Allow users to customize password composition.

• Evaluate password strength.

• Provide a simple and user-friendly graphical interface.

• Help users improve password security.

3. System Architecture

The system is divided into two main modules:

1. Backend Module

2. Frontend Module

The backend handles the password generation logic and strength checking.

The frontend provides the graphical user interface for user interaction.

Flow:

User → Frontend (GUI) → Backend Logic → Generated Password + Strength

4. Modules Used in the Project

4.1 Tkinter Module

Tkinter is Python's built-in library used to create Graphical User Interfaces (GUI).

It is used to:

• Create the main application window

• Add buttons, labels, text boxes

• Handle user input

• Display password results

Tkinter allows the program to run as a desktop application rather than a command-line program.

4.2 Secrets Module

The secrets module is used to generate cryptographically secure random numbers. It is safer than the normal random module and is recommended for security-related applications such as password generation.

Functions used:

• secrets.choice()

• secrets.SystemRandom().shuffle()

Purpose: Ensure passwords are unpredictable and secure.

4.3 String Module

The string module provides predefined character sets.

Examples:

• ascii_uppercase → A–Z

• ascii_lowercase → a–z

• digits → 0–9

• punctuation → special characters

This module helps build the character pool used for password generation.

4.4 Pyperclip Module

The pyperclip module allows the program to interact with the system clipboard.

Function used:

pyperclip.copy(password)

Purpose:

It allows users to copy the generated password instantly and paste it anywhere.

5. Backend Module

The backend module (password_generator.py) contains the main logic of the application. It is responsible for generating passwords and checking password strength.

5.1 Password Generation Function

generate_password()

Purpose:

Generates a password based on user-selected options.

User options include:

• Uppercase letters

• Lowercase letters

• Numbers

• Symbols

• Excluding ambiguous characters (O, 0, l, I, 1)

Steps performed:

1. Create character pools

2. Remove ambiguous characters if selected

3. Ensure required character types are included

4. Fill remaining characters randomly

5. Shuffle the password

6. Return the final password

5.2 Password Strength Evaluation

check_strength()

This function evaluates how strong the password is.

It checks:

• Password length

• Uppercase characters

• Lowercase characters

• Numbers

• Symbols

Strength levels:

Weak – Low security

Moderate – Medium security

Strong – High security

6. Frontend Module (User Interface)

The frontend is developed using Tkinter and provides the graphical interface for users to interact with the system.

Main components:

• Main window

• Background image

• Toggle buttons

• Password generation button

• Password display box

• Copy password button

• Password strength indicator

7. Features of the System

• Secure password generation

• Customizable password options

• Password strength evaluation

• Background styled GUI

• Toggle buttons for character selection

• Copy password functionality

• Easy-to-use interface

8. Advantages of the System

• Generates strong passwords automatically

• Easy for users to operate

• Improves security awareness

• Saves time in creating passwords

• Reduces risk of weak passwords

9. Limitations

• Runs only on desktop systems

• Password history is not stored

• No cloud synchronization

10. Future Enhancements

• Add encrypted password storage

• Integrate a password manager

• Add light/dark theme options

• Export passwords securely

• Develop a mobile version

11. Conclusion

The Secure Password Generator demonstrates how software can improve digital security.

By combining secure password generation with a user-friendly interface, the system helps users create strong

and reliable passwords for their accounts.

The project also demonstrates the use of Python libraries such as Tkinter, secrets, string, and pyperclip

to build a functional security tool.